

Marissa Ratschki

CMPT 308 Lab 08

## **PART A:**

### **ROLE CREATION +GRANT/REVOKE:**

```
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'advisor_role') THEN
        CREATE ROLE advisor_role;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'registrar_role') THEN
        CREATE ROLE registrar_role;
    END IF;
END $$;

GRANT SELECT ON lab8_students, lab8_courses, lab8_enrollments TO advisor_role;

GRANT SELECT ON lab8_students, lab8_courses, lab8_enrollments TO registrar_role;
GRANT INSERT ON lab8_enrollments TO registrar_role;
GRANT UPDATE ON lab8_courses TO registrar_role;

REVOKE DELETE ON lab8_enrollments FROM registrar_role;
```

### **PRIVILEGE REPORT QUERY:**

```
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('advisor_role', 'registrar_role')
    AND table_name IN ('lab8_students', 'lab8_courses', 'lab8_enrollments')
ORDER BY grantee, table_name, privilege_type;
```

### **OUTPUT:**

|                  |                    |          |
|------------------|--------------------|----------|
| "advisor_role"   | "lab8_courses"     | "SELECT" |
| "advisor_role"   | "lab8_enrollments" | "SELECT" |
| "advisor_role"   | "lab8_students"    | "SELECT" |
| "registrar_role" | "lab8_courses"     | "SELECT" |
| "registrar_role" | "lab8_courses"     | "UPDATE" |
| "registrar_role" | "lab8_enrollments" | "INSERT" |
| "registrar_role" | "lab8_enrollments" | "SELECT" |
| "registrar_role" | "lab8_students"    | "SELECT" |

**COMMENT:**

*The advisor\_role is read-only because it has SELECT privileges on the students, courses, and enrollments tables. The registrar\_role can also view those tables, but it has additional INSERT permission on lab8\_enrollments and UPDATE permission on lab8\_courses. The registrar\_role does not have DELETE permission on enrollments, which limits it from removing registration records*

**PART B:**

```
CREATE OR REPLACE FUNCTION lab8_register_student(
  p_student_id INT,
  p_course_id TEXT
)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
  v_student_name TEXT;
  v_course_title TEXT;
  v_capacity INT;
  v_enrolled_count INT;
BEGIN
  SELECT student_name
  INTO v_student_name
  FROM lab8_students
  WHERE student_id = p_student_id;

  IF NOT FOUND THEN
    RETURN 'ERROR: student ' || p_student_id || ' does not exist.';
  END IF;

  SELECT title, capacity, enrolled_count
  INTO v_course_title, v_capacity, v_enrolled_count
  FROM lab8_courses
  WHERE course_id = p_course_id
  FOR UPDATE;

  IF NOT FOUND THEN
    RETURN 'ERROR: course ' || p_course_id || ' does not exist.';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM lab8_enrollments
```

```

WHERE student_id = p_student_id
AND course_id = p_course_id
) THEN
RETURN 'ERROR: ' || v_student_name || ' is already enrolled in ' || p_course_id || ';
END IF;

IF v_enrolled_count >= v_capacity THEN
RETURN 'ERROR: course ' || p_course_id || ' is full.';
END IF;

INSERT INTO lab8_enrollments (student_id, course_id)
VALUES (p_student_id, p_course_id);

UPDATE lab8_courses
SET enrolled_count = enrolled_count + 1
WHERE course_id = p_course_id;

RETURN 'SUCCESS: enrolled ' || v_student_name || ' in ' || p_course_id || ' - ' ||
v_course_title || ';
END;
$$;

```

Test 1: SELECT lab8\_register\_student(1, 'CMPT308') AS result;  
Output: SUCCESS: enrolled Ana in CMPT308 - Database Management.

Test 2: SELECT lab8\_register\_student(1, 'CMPT308') AS result;  
Output: ERROR: Ana is already enrolled in CMPT308.

Test 3: SELECT lab8\_register\_student(25, 'CMPT308') AS result;  
Output: ERROR: student 25 does not exist.

#### **COMMENT:**

*This function centralizes the course registration workflow inside the database. It checks whether the student exists, whether the course exists, whether the student is already enrolled, and whether the course has open seats before making any changes. This is useful as the same rules are enforced consistently, even if different users or applications try to enroll students.*

#### **PART C:**

```

CREATE OR REPLACE FUNCTION lab8_log_enrollment_insert()
RETURNS TRIGGER
LANGUAGE plpgsql

```

```

AS $$
BEGIN
    INSERT INTO lab8_enrollment_audit (
        action_type,
        student_id,
        course_id,
        action_time
    )
    VALUES (
        'INSERT',
        NEW.student_id,
        NEW.course_id,
        CURRENT_TIMESTAMP
    );

    RETURN NEW;
END;
$$;

```

```

CREATE TRIGGER lab8_enrollment_insert_audit_trg
AFTER INSERT ON lab8_enrollments
FOR EACH ROW
EXECUTE FUNCTION lab8_log_enrollment_insert();

```

Test 1: SELECT lab8\_register\_student(2, 'CMPT101') AS result;  
Output: SUCCESS: enrolled Tyler in CMPT101 - Intro to Cybersecurity.

Test 2 (course full): SELECT lab8\_register\_student(3, 'CMPT101') AS result;  
Output: ERROR: course CMPT101 is full.

```

SELECT audit_id, action_type, student_id, course_id, action_time
FROM lab8_enrollment_audit
ORDER BY action_time, audit_id;

```

Output:

|   |          |   |           |                             |
|---|----------|---|-----------|-----------------------------|
| 1 | "INSERT" | 2 | "CMPT101" | "2026-06-16 18:01:32.26074" |
|---|----------|---|-----------|-----------------------------|

#### **COMMENT:**

*This audit logic is a good fit for a trigger because it should happen automatically whenever a new enrollment is inserted. If audit rows had to be inserted manually, a*

*user could forget to record the event. The trigger makes the audit log reliable because it runs as part of the operation itself.*

#### **PART D:**

Enrollment list query:

```
SELECT
    s.student_name,
    e.course_id,
    c.title AS course_title,
    e.enrolled_at
FROM lab8_enrollments e
JOIN lab8_students s
    ON e.student_id = s.student_id
JOIN lab8_courses c
    ON e.course_id = c.course_id
ORDER BY e.enrolled_at, s.student_name, e.course_id;
```

Output:

|         |           |                          |                              |
|---------|-----------|--------------------------|------------------------------|
| "Ana"   | "CMPT308" | "Database Management"    | "2026-06-16 17:52:13.132235" |
| "Tyler" | "CMPT101" | "Intro to Cybersecurity" | "2026-06-16 18:01:32.26074"  |

Seats remaining query:

```
SELECT
    course_id,
    title,
    capacity,
    enrolled_count,
    capacity - enrolled_count AS seats_remaining
FROM lab8_courses
ORDER BY course_id;
```

Output:

|           |                          |   |   |   |
|-----------|--------------------------|---|---|---|
| "CMPT101" | "Intro to Cybersecurity" | 1 | 1 | 0 |
| "CMPT200" | "Internetworking"        | 3 | 0 | 3 |
| "CMPT308" | "Database Management"    | 2 | 1 | 1 |

Audit log query:

```
SELECT
    audit_id,
    action_type,
    student_id,
```

```
course_id,  
action_time  
FROM lab8_enrollment_audit  
ORDER BY action_time, audit_id;
```

Output:

```
1      "INSERT"      2      "CMPT101"  "2026-06-16 18:01:32.26074"
```

Privilege report query:

```
SELECT grantee, table_name, privilege_type  
FROM information_schema.role_table_grants  
WHERE grantee IN ('advisor_role', 'registrar_role')  
AND table_name IN ('lab8_students', 'lab8_courses', 'lab8_enrollments')  
ORDER BY grantee, table_name, privilege_type;
```

Output:

```
"advisor_role""lab8_courses"      "SELECT"  
"advisor_role""lab8_enrollments"  "SELECT"  
"advisor_role""lab8_students"     "SELECT"  
"registrar_role"      "lab8_courses"      "SELECT"  
"registrar_role"      "lab8_courses"      "UPDATE"  
"registrar_role"      "lab8_enrollments"  "INSERT"  
"registrar_role"      "lab8_enrollments"  "SELECT"  
"registrar_role"      "lab8_students"     "SELECT"
```

## **PART E:**

E1: Authentication verifies who a user is, such as through a login, password, or other credentials. Authorization determines what a user is allowed to do after their identity is authenticated.

E2: Least privilege, both in and outside of the database context, is important because users and roles should receive the minimum permissions needed for their responsibilities. This reduces the risk of accidental changes, misuse, or damage from a compromised account.

E3: A stored procedure or function is called directly to perform a specific task, such as registering a student for a course. A trigger runs automatically in response to a database event.

E4: Too much hidden trigger logic can make a system harder to debug because data may change automatically without being obvious from the original SQL command. If many triggers are running behind the scenes, it can be difficult to trace where an error resulted from or why it occurred.